HOWTO Install PHP as CGI with Apache's suEXEC Feature

Dominik Ruf

<dodolin@debianhowto.de>

2003-06-15

Revision History

Revision 1.0	2003-06-15	dodolin
Added Section 2.3 (fixed TODO). First public release.		
Revision 0.7	2003-06-14	dodolin
Added Sections 2 and 3. Still some TODOs.		
Revision 0.5	2003-06-01	dodolin
Startrelease [BETA!]		

Installing PHP as an Apache DSO Module is most often not the way to go, especially if there are other web users on your machine and/or if you're concerned about security. This HOWTO describes a more secure way of installing PHP as a CGI binary where the PHP parser is located outside the web tree and gets called via Apache's suEXEC Feature.

Table of Contents

Introduction 1
- Copyright and License 2
- Disclaimer 2
- Feedback 2

Installation 3
- Prerequisites 3
- Installing php4-cgi 3
- Build a custom php4-cgi Debian package yourself 3

Configuration 5
- Apache Configuration 5
- Testing the installation 6

Introduction

Installing PHP as an Apache DSO Module [http://www.php.net/manual/en/security.apache.php] is most often not the way to go, especially if there are other web users on your machine and/or if you're concerned about security. This

HOWTO describes a more secure way of installing PHP as a CGI binary [http://www.php.net/manual/en/security.cgi-bin.php] where the PHP parser is located outside the web tree and gets called via Apache's suEXEC Feature [http://httpd.apache.org/docs/suexec.html].

Important

For a better understanding of the background and the implications of this setup I highly recommend reading the above URIs including the user-comments in the PHP manual.

The only real downside to this setup is that you will now have to put a line similar to:

```
#!/usr/bin/php4
```

as the first line of every file containing PHP tags and you'll have to make them executable - at least for the user they'll run on later. If you feel comfortable with this, just read on...

If these disadvantages are not acceptable for you, you may want to take a look at pookey's website about Securing PHP on Debian Linux [http://www.pookey.co.uk/php-secure.php] instead. He uses the binfmt_misc kernel module and therefor doesn't need to use shebangs (#!/usr/bin/php4). However, I didn't choose pookey's way because I'm having a "bad feeling" using an additional kernel module for every PHP request.

Copyright and License

This document, *HOWTO install PHP as CGI with Apache's suEXEC Feature*, is copyrighted (c) 2003 by *Dominik Ruf*. Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.1 or any later version published by the Free Software Foundation; with no Invariant Sections, with no Front-Cover Texts, and with no Back-Cover Texts. A copy of the license is available at http://www.gnu.org/copyleft/fdl.html [http://www.gnu.org/copyleft/fdl.html].

Linux is a registered trademark of Linus Torvalds.

Disclaimer

No liability for the contents of this document can be accepted. Use the concepts, examples and information at your own risk. There may be errors and inaccuracies, that could be damaging to your system. Proceed with caution, and although this is highly unlikely, the author(s) do not take any responsibility.

All copyrights are held by their respective owners, unless specifically noted otherwise. Use of a term in this document should not be regarded as affecting the validity of any trademark or service mark. Naming of particular products or brands should not be seen as endorsements.

Feedback

Feedback is most certainly welcome for this document. Send your additions, comments and criticisms to the following email address : <dodolin@debianhowto.de>.

Installation

Prerequisites

Of course, you'll need a working installation of Apache. If not already done yet, **apt-get install apache**.

Installing php4-cgi

As described in the PHP Manual [http://www.php.net/manual/en/security.cgi-bin.php] ("Case 4: PHP parser outside of web tree") PHP has to be compiled with the --enable-discard-path configure option which is not the case with Debians php4-cgi package. Debians php4-cgi package also has mysql-support disabled. For your convenience I've created an **apt-get**able php4-cgi package. Just add the following line to your /etc/apt/sources.list:

```
deb http://deb.debianhowto.de testing php_cgi
```

and **apt-get update ; apt-get install php4-cgi**. If you want to know in detail what I changed exactly here's an excerpt from debian/changelog:

```
php4 (4:4.1.2-10.dodolin.0) stable; urgency=low

  * Changed ./configure Options (for php4-cgi):
    - --enable-force-cgi-redirect into --enable-discard-path
    - --without-mysql into --with-mysql=shared,/usr

 -- Dominik Ruf <dodolin@debianhowto.de>  Mon,  2 Jun 2003 22:10:09 +0200
```

Build a custom php4-cgi Debian package yourself

In case you don't want to use my packages or if you need additional PHP modules compiled in, I'll now describe the steps needed to build your own custom php4-cgi package for your Debian box.

First, we need the sources to be modified and the required development files to build the package. This could be easily accomplished with the tools Debian has to offer:

```
$ apt-get source php4
```

```
# apt-get build-dep php4
```

Now, open the file `php4-<version>/debian/rules` in your favourite editor and search for the section beginning with:

```
configure-cgi-stamp:
```

and edit the configure-options after

```
../configure
```

to suit your needs. In our case, we need to change --enable-force-cgi-redirect into --enable-discard-path. I also changed --without-mysql into --with-mysql=shared,/usr, but this is optional for our task to install PHP as CGI via suEXEC.

To avoid that our custom php4-cgi package gets overridden with Debian's default php4-cgi when you run **apt-get upgrade** the next time it's recommended to increase the minor version number of the package. (We also could set the package status to "hold" instead but IMHO this is not as elegant as increasing the minor version number.) This is done by editing the file `php4-<version>/debian/changelog` :

Important

This file has to follow special syntax rules (for details, see http://www.debian.org/doc/debian-policy/ch-miscellaneous.html#s-dpkgchangelog [http://www.debian.org/doc/debian-policy/ch-miscellaneous.html#s-dpkgchangelog]) or things will break. Therefor, the easiest way is to copy the last entry (which is at the *top* of the file) and edit it accordingly.

In my example the top of the file looks like this:

```
php4 (4:4.1.2-10.dodolin.0) stable; urgency=low

  * Changed ./configure Options (for php4-cgi):
    - --enable-force-cgi-redirect into --enable-discard-path
    - --without-mysql into --with-mysql=shared,/usr

 -- Dominik Ruf <dodolin@debianhowto.de>  Mon,  2 Jun 2003 22:10:09 +0200

php4 (4:4.1.2-6) stable; urgency=low

[other entries ...]
```

Note: I increased the minor version number not only in one but in several numbers to be in a safe place in case there are some security updates (which increase the minor version number in one).

After that, we're ready to build our package(s). Change into the directory `php-<version>` and execute the following command: **dpkg-buildpackage -us -uc -rfakeroot**. If everything went well you'll find your custom php4-* Debian packages one directory level up. Finally, install your php4-cgi package with **dpkg -i <filename>** (e.g. **dpkg -i php4-cgi_4.1.2-10.dodolin.0_i386.deb**).

Configuration

Apache Configuration

After installing an appropriate php4-cgi package we have to customize the apache configuration file `/etc/apache/httpd.conf`. First, we have to tell apache that it should treat every file with extension .php as a cgi-script. Uncomment the following line in your `httpd.conf` and change it accordingly to fit your needs, e.g.:

```
AddHandler cgi-script .php .cgi .sh .pl
```

Now, every <Directory> section where CGIs should be enabled needs:

```
Options +ExecCGI
```

Examples

Tip

Recommended reading: Using suEXEC [http://httpd.apache.org/docs/suexec.html#usage] (Apache Documentation)

To enable PHP for ~user requests, change the "Options" directive of the default `httpd.conf` as follows:

```
#
# Control access to UserDir directories.  The following is an example
# for a site where these directories are restricted to read-only.
#
<Directory /home/*/public_html>
    AllowOverride FileInfo AuthConfig Limit
    #Options MultiViews Indexes SymLinksIfOwnerMatch IncludesNoExec
    Options MultiViews Indexes SymLinksIfOwnerMatch ExecCGI
    <Limit GET POST OPTIONS PROPFIND>
        Order allow,deny
        Allow from all
    </Limit>
    <Limit PUT DELETE PATCH PROPPATCH MKCOL COPY MOVE LOCK UNLOCK>
```

```
        Order deny,allow
        Deny from all
    </Limit>
</Directory>
```

To allow PHP (and maybe other - see your "AddHandler" directive) scripts for a specific virtual host to be run under another user - say: www-user - use something like this in your vhost configuration:

```
<VirtualHost *>
    ServerName www.dodolin.de
    ServerAlias dodolin.de
    DocumentRoot /var/www/vhosts/www.dodolin.de/htdocs
    User www-user
    Group www-user

    <Directory /var/www/vhosts/www.dodolin.de/htdocs/>
        Options +ExecCGI
        Order Deny,Allow
        Allow from all
    </Directory>
</VirtualHost>
```

Important

The default suEXEC shipped with Debian's Apache is compiled with --suexec-docroot=/var/www [http://httpd.apache.org/docs/suexec.html#install]. That meens that this only works if you put your vhosts somewhere under `/var/www` as can be seen in this example.

Testing the installation

First, make sure that your suEXEC is working in general by having a look at your error log (default: `/var/log/apache/error.log`) upon startup of Apache (see http://httpd.apache.org/docs/suexec.html#enable [http://httpd.apache.org/docs/suexec.html#enable]). If you see a line like this:

```
[notice] suEXEC mechanism enabled (wrapper: /path/to/suexec)
```

everything is fine.

Next, create a test-script under a ~user directory, e.g. as user bofh, create `~/bofh/public_html/phpinfo.php` with the following contents:

```
#!/usr/bin/php4

<?php phpinfo(); ?>
```

Don’t forget to make the file executable! As it runs under the user bofh, permissions of 700 are sufficient (**chmod 700 ~/bofh/public_html/phpinfo.php**). Then access this file with your favourite browser, e.g. **lynx http://localhost/~bofh/phpinfo.php** and watch your `/var/log/apache/suexec.log`. If you see something similar to this

```
[2003-06-14 16:44:07]: info: (target/actual) uid: (bofh/bofh) gid: (bofh/bofh) cmd: phpinfo.php
```

everything is fine and working. If it’s not working try using the logfiles to find out what’s going wrong (see http://httpd.apache.org/docs/suexec.html#debug [http://httpd.apache.org/docs/suexec.html#debug]).